Customer Support Automation Pipeline: Final Report

Executive Summary

This report summarizes the design, implementation, and evaluation of an LLM-driven customer support automation pipeline. The system triages incoming customer queries (intents), retrieves relevant resolution steps (RAG), drafts personalized responses, and escalates to human agents when rules are triggered. Built primarily using the NVIDIA NIM `nemotron-3-super-120b-a12b` model for classification and drafting, and evaluated rigorously against human-labeled ground truth, the project highlights significant findings regarding few-shot LLM limitations on imbalanced datasets and the persistent value of keyword-based heuristics.

1. Problem Framing

For the AmazonHelp support automation pipeline, a "good" agent is not solely defined by high absolute intent accuracy, but rather by its ability to safely handle high-stakes customer interactions. Specifically, "good" means correctly routing time-sensitive issues (like unauthorized account access or active delivery failures), not making false promises or legally binding statements in drafted replies, and reliably catching genuine escalation-worthy anger to ensure human intervention. A system that correctly answers 90% of routine queries but mishandles a severe safety complaint is a failure for the brand.

To maintain strict scope and safety, several features were deliberately excluded from this prototype:

- **No Learned Escalation Classifier:** A deterministic rule engine (D5) was chosen over an ML classifier because 189 golden examples are insufficient for reliable training, and rules provide 100% auditable, zero-latency logic. However, the subsequent 3.1% recall finding revealed that these strict keyword rules are too rigid to catch nuanced human frustration.
- **No Fine-Tuned Classifier:** We utilized a few-shot prompting approach rather than fine-tuning a model to reduce pipeline complexity and deployment overhead. Given the low 38.6% accuracy, relying solely on few-shot prompting for highly imbalanced, ambiguous data proved insufficient.
- **No Live Production Deployment or Real-Time Streaming:** The system runs as a batch analysis and dashboard layer over historical data. Building live API ingestion and webhook responses was excluded to prioritize evaluation rigor and methodology over infrastructure engineering.
- **No Multi-Brand Generalization:** The intent taxonomy and escalation rules were strictly tailored to AmazonHelp to ensure high relevance. (The Banking77 cross-domain test was purely an exploratory probe, not a design goal).

2. System Architecture

The pipeline processes incoming Twitter customer support data (AmazonHelp) via four stages:

1. Intent Classification: A few-shot prompt (2 examples per intent) instructs `nvidia/nemotron-3-super-120b-a12b` to classify tweets into 11 distinct Amazon-specific intents.

2. Retrieval (RAG): Uses `sentence-transformers/all-MiniLM-L6-v2` embeddings and a FAISS IndexFlatIP to search 8,000 resolved historical threads. The search is filtered first by the predicted intent, then ranked by semantic similarity.

3. Response Drafting: `nemotron-3-super-120b-a12b` generates a draft reply grounded in the retrieved historical thread and the customer's query.

4. Escalation Engine: A deterministic, 8-rule engine flags interactions needing human intervention (e.g., negative sentiment, legal threats, multiple failures).

3. Evaluation Methodology (Honest Reporting)

Early metrics suffered from label leakage: evaluating the NIM classifier against an auto-labeled dataset generated by a keyword heuristic. To ensure honest reporting, the final evaluation was conducted against a 189-row human-labeled golden set (`data/golden_labeled.csv`). This set reflects the true, imbalanced distribution of real-world interactions (54% of queries fall into a catch-all `general_complaint_or_feedback` class).

4. Key Findings & Final Metrics

4.1 Intent Classification (Golden Set)

- Accuracy: 38.6%
- Majority Baseline: 54.0%
- Keyword Baseline: 52.9%
- Zero-shot Baseline: 46.0%

Finding: The few-shot LLM classifier underperforms both the majority-class baseline and a simple keyword heuristic. The LLM aggressively over-predicts specific, narrow intents (e.g., `delivery_issue`, `product_issue`) when presented with ambiguous or broad customer frustration (the `general_complaint_or_feedback` majority class). It "hallucinates" specific intents based on minor semantic signals, while the keyword heuristic safely defaults to the catch-all class. Additionally, the model is highly miscalibrated, showing high confidence (>90%) on incorrect predictions.

4.2 Information Retrieval

- Hit@1: 42.9%
- Hit@3: 61.9%
- Hit@5: 70.9%

Finding: The FAISS index effectively retrieves similar historical threads, but because the RAG filter relies on the *predicted intent* from the classifier, the overall system accuracy is severely bottlenecked. A wrong intent prediction (61.4% of the time) guarantees retrieval from the wrong pool of historical resolutions.

4.3 Escalation Engine

- Precision: 50.0%
- Recall: 3.1%

Finding: The deterministic ruleset severely under-flags escalations compared to human judgment. While avoiding false positives (good precision), the rules fail to capture the nuanced frustration that triggers human escalation. The trade-off was accepted to maintain a zero-latency, zero-cost, auditable rule engine rather than overfitting a binary classifier to the small golden set.

4.4 Response Quality (LLM-as-Judge)

Drafts were evaluated by a cross-family judge model (`openai/gpt-oss-20b`) to eliminate same-family bias, scoring on a 1-5 scale:

- Relevance: 4.78
- Empathy: 4.59
- Actionability: 4.26
- Conciseness: 4.56
- Overall: 4.41

Finding: When the LLM generates a response, it produces highly empathetic, relevant, and well-structured replies. Human agreement on the `actionability` and `overall` metrics is strong (74-77% within ±1 point). However, humans rate the model much more harshly on `relevance` and `empathy` than the LLM judge does.

4.5 Cross-Domain Generalization (Banking77)

- In-Domain Accuracy (Golden): 38.6%
- Cross-Domain Accuracy: 40.0%

Finding: Testing the Amazon-trained system against banking queries mapped to equivalent intents yielded 40.0% accuracy. The fact that cross-domain performance effectively matches in-domain performance highlights that the model is relying heavily on superficial keyword mapping rather than deep semantic understanding of the customer's core intent.

5. What's Misleading About My Headline Number

Initially, the reported intent classifier accuracy was 64.6%. This number was highly misleading due to the following factors:

- **Label Leakage:** The 64.6% accuracy was evaluated against an auto-labeled dataset (`labelled_eval.jsonl`) generated using the exact same keyword heuristic as Baseline 1. This artificially inflated the system's performance, as it effectively tested whether the LLM could predict the heuristic's output, not the ground truth.
- **Confidence Miscalibration:** The model displayed severe overconfidence on incorrect predictions. The accuracy was lowest (around 35%) for predictions made with high confidence (>90%), indicating that the model's confidence scores are unreliable.

- **Retrieval Bottleneck:** Since the RAG filter relies directly on the predicted intent, an incorrect intent prediction guarantees retrieval from the wrong pool of historical resolutions. The 38.6% true accuracy means ~61% of queries are bottlenecked by wrong-intent filtering before retrieval even begins.
- **Cross-Domain Gap Reversal:** Initial findings showed a massive 24.6% gap between in-domain and cross-domain performance. However, evaluating on the golden set revealed that true in-domain accuracy is 38.6%, making the cross-domain accuracy (40.0%) effectively equal. The model doesn't just fail to generalize—it performs poorly in both domains, largely relying on superficial keyword associations.

6. Top 5 Failure Modes

1. general_complaint_or_feedback Over-Classification:

- **Example Tweet:** "@AmazonHelp Really bad experience. Frustrated. Paid for fast shipping and it's delayed again."
- **Predicted Intent:** `shipping_delay_complaint` (True Label: `general_complaint_or_feedback`)
- **Hypothesis:** The model heavily over-predicts specific intents when it sees strong keywords ("shipping", "delayed") even if the core intent is a general expression of frustration.

2. Confidence Miscalibration:

- **Example Tweet:** "@AmazonHelp with an amazon fire tablet, can I tap on a photo to see it full screen?"
- **Predicted Intent:** `device_and_digital_support` (True Label: `general_complaint_or_feedback` / `other`)
- **Hypothesis:** The model equates keyword matching with certainty. The presence of specific entities ("fire tablet") leads to >95% confidence despite predicting the wrong label.

3. Escalation Recall Failure:

- **Example Tweet:** "@AmazonHelp I am so angry right now! My package was stolen and your reps hung up on me. I want a manager NOW."
- **Prediction:** Not escalated.
- **Hypothesis:** The deterministic 8-rule escalation engine relies on exact keyword triggers (e.g., specific swear words or literal "sue you") and misses nuanced, high-intensity frustration.

4. Retrieval Bottleneck from Wrong-Intent Filtering:

- **Example Tweet:** "@AmazonHelp my kindle screen froze."
- **Predicted Intent:** `product_issue` (instead of `device_and_digital_support`)
- **Hypothesis:** Because the classifier predicts `product_issue`, FAISS only searches the `product_issue` index subset. The best historical resolutions for Kindle freezing are hidden in the `device_and_digital_support` index.

5. shipping_delay_complaint Cross-Domain Collapse:

- **Example Query (Banking77):** "My new card hasn't arrived yet, it's been weeks."

- **Predicted Intent:** `delivery_issue` (True Mapped Label: `shipping_delay_complaint`)
- **Hypothesis:** The model lacks deep semantic understanding. "Card hasn't arrived" sounds like a delivery failure rather than a shipping delay, exposing the model's reliance on Amazon-specific phrasing.

7. Next Steps

1. **Fix Escalation Recall:** Replace the rigid deterministic rules with a hybrid approach—a lightweight LLM prompt specifically designed to assess emotional intensity and escalation necessity, acting as an override layer.

2. **Address general_complaint_or_feedback Over-Prediction:** Refine the classifier's prompt to be more conservative. Explicitly instruct the model to default to `general_complaint_or_feedback` unless specific actionable criteria are met for the narrow intents.

3. **Expand and Rebalance Golden Set:** The current 189-row golden set is highly skewed (54% `general_complaint_or_feedback`). Hand-labeling an additional 300-500 tweets, specifically oversampling minority classes, will provide a more robust evaluation metric for weighted F1.

8. Design Decisions & Trade-offs

- **FAISS Exact Search vs. HNSW:** With only 8,000 vectors (384 dims), exact search (IndexFlatIP) completes in <10ms. HNSW approximate search was deemed unnecessary complexity.
- **11 Intents vs. 77:** Initial clustering revealed that 77 intents were too granular for the dataset. The taxonomy was collapsed to 11 pragmatic, actionable intents.
- **Deterministic Rules vs. ML Escalation:** Opted for explainability and zero-cost over a learned model due to the small (189 rows) human-labeled dataset, despite poor recall.

9. Conclusion

This project demonstrates that while large language models excel at drafting empathetic, fluent text (Generative/Drafting stage), they are not necessarily superior to simple heuristic rules for classification tasks on heavily skewed, ambiguous real-world data. The keyword baseline's victory over the few-shot 120B parameter model is the most crucial finding: deploying complex, expensive LLMs for rigid classification tasks without rigorous golden-set validation can lead to degraded performance and false confidence. Future iterations should use the LLM to extract semantic features (entities, sentiment) while leaving the routing/classification logic to trained lightweight models or heuristics.